
MNEMOS 32

A Multi-Core RV32IMA+P SoC with 5-Stage Pipelining and L1 Cache Hierarchy

Manthankumar Bagade, Sarvesh Joshi
Abhigyan Sinha, Shivansh Mittal

Department of Computer Science and Engineering
Indian Institute of Technology Guwahati

March 2026

1. System Description

This system architecturally evolves a baseline 3-stage RV32I processor into a high-performance, multi-core RV32IMA. Each constituent core implements a 5-stage pipeline integrated with dedicated L1 instruction and data caches, complemented by dynamic branch prediction to optimize instruction throughput. Scalar arithmetic bottlenecks are eliminated via a hardware multiplier (RV32M), while a centralized bus arbiter facilitates seamless inter-core communication. To support robust multithreaded execution, the SoC utilizes the RV32A extension for hardware-level atomicity, managing shared-data integrity through mutexes and exclusive memory monitors utilizing bus-snooping techniques. Furthermore, the datapath is augmented with the RV32P extension, leveraging sub-word parallelism (SIMD) to accelerate matrix-intensive math for quantized neural inference by executing four parallel 8-bit operations per clock cycle

2. Use Cases

- **High-Performance Synchronization:** Utilizes RV32A `lr.w/sc.w` primitives to facilitate lock-free concurrency. This enables multiple cores to safely manage shared data structures, such as concurrent streaming queues, without the performance bottlenecks of global memory bus locking.
- **Parallel Workload Distribution:** The multi-core, 5-stage pipelined architecture allows for true parallel execution of multithreaded C/C++ programs. With integrated L1 caches and dynamic branch prediction, the SoC can distribute independent computational tasks across cores, maximizing instruction throughput while minimizing pipeline stalls during complex algorithm execution.
- **Accelerated Neural Inference:** Combines the RV32M hardware multiplier with RV32P 8-bit packed-SIMD operations to process quantized machine learning matrix arrays. By executing four parallel operations per clock cycle, the hardware significantly outperforms standard software-based scalar arithmetic.

3. FPGA Relevance

- **Cycle-Accurate Concurrency Validation:** Validating the intricate timing of inter-core bus-snooping logic and atomic memory reservation states (RV32A) requires exact clock-cycle determinism. FPGA physical synthesis provides a true hardware-in-the-loop environment,

allowing for the detection of sub-cycle race conditions that software simulators often fail to replicate.

- **Architectural Datapath Manipulation:** FPGAs enable the physical synthesis of specialized hardware blocks, such as multi-cycle sequential multipliers (RV32M) and the dynamic restructuring of ALU carry-chains. This flexibility allows for the implementation of 8-bit packed-SIMD execution (RV32P).
- **Real-Time Hardware Telemetry:** By integrating a memory-mapped UART peripheral, the system establishes a direct telemetry link to a laptop terminal. This interface allows for the live serialization of critical hardware metrics, including instructions per cycle (IPC), L1 cache hit/miss rates, and internal register snapshots, providing high-visibility verification of multi-core performance.

4. System Scope

Minimum System (Commitment): The minimum goal is to create a working multi-core 5-stage processor with support for the RV32IMA Instruction Set Architecture.

- Convert the existing 3-stage pipeline into a 5-stage one significantly improving pipeline latency and reducing CPI.
- Implement a functional multi-cycle sequential hardware multiplier to support the ISA of the RV32M extension.
- Implement a bus-arbiter capable of managing a shared BRAM for multiple cores and minimizing pipeline stalls.
- Add support for the RV32A ISA which includes atomic operations and mutexes enabling functioning of multi-threaded algorithms.

Goal System: Beyond the primary system commitments, the goal system aims to maximize architectural efficiency and observability through the following high-performance enhancements:

- **Dynamic Branch Prediction:** Integrate per-core branch target buffers and prediction logic to resolve control-flow hazards early in the 5-stage pipeline, effectively minimizing execution stalls during complex branching operations.
- **Integrated L1 Cache Hierarchy:** Incorporate dedicated L1 instruction and data caches for each core to exploit spatial and temporal data locality, significantly reducing average memory access latency compared to direct BRAM transactions.
- **High-Visibility UART Telemetry:** Implement a memory-mapped UART transmitter to enable real-time hardware profiling. This establishes a serial link to an external laptop terminal for the live monitoring of instruction throughput, cache hit rates, and internal register snapshots.

Stretch Goal:

- **Hardware Performance Monitoring:** Implement the Zicsr extension, specifically the `mcycle` and `minstret` Control and Status Registers, to enable cycle-accurate calculation of Instructions Per Cycle (IPC) and other critical hardware-level profiling metrics.
- **High-Throughput SIMD Integration:** Fully integrate the RV32P ISA extension into the execution unit to support 8-bit sub-word parallelism, providing the necessary datapath width to accelerate large-scale matrix-multiplication kernels.
- **Heterogeneous Neural Training:** Augment the datapath with a functional subset of the RV32F (Single-Precision Floating-Point) extension to support on-chip neural network training; this framework will leverage multi-core thread distribution alongside RV32P SIMD

acceleration to execute a complete end-to-end neural inference and learning cycle.

5. Design Goals

- **Latency (cycles):** Targeted at maintaining a baseline $CPI \approx 1.0$ for non-conflicting integer operations. The 5-stage pipeline will be designed to dynamically stall strictly for multi-cycle hardware multiplication, unresolved data hazards, and inter-core memory bus arbitration delays.
- **System Throughput:** Facilitate native execution of 32-bit multiplication operations in hardware to eliminate scalar math bottlenecks. Aggregate system throughput is scaled by enabling the simultaneous execution of parallel instruction streams across multiple independent processor cores.
- **Resource Optimization (LUT/BRAM):** Manage architectural complexity to constrain total logic utilization to $\leq 50\%$ of available FPGA LUTs for the multi-core configuration. Effectively map compiled multithreaded C-binaries and L1 cache structures into physical BRAM resources.
- **Execution Correctness:** Achieve a 100% success rate on rigorous multithreaded benchmarking C-programs. Verification will specifically focus on inter-core atomic lock integrity via the RV32A extension and signed/unsigned multiplication boundary conditions.

6. Implementation Approach

The implementation follows a modular RTL design strategy, focusing on scaling the core complexity and memory hierarchy while maintaining system-wide coherency:

- **Hardware Math Integration:** A specialized `mult_div.v` execution unit will be integrated into the datapath to provide native hardware support for the RV32M extension.
- **Pipeline Refactoring:** The existing 3-stage architecture will be refactored into a 5-stage pipeline (IF, ID, EX, MEM, WB) to improve maximum operating frequency and enable structured hazard management.
- **Inter-Core Arbitration:** A centralized `bus_arbiter.v` module will be developed to mediate memory access requests between multiple cores, ensuring deterministic bus access and starvation-free communication.
- **Memory Hierarchy Optimization:** Independent L1 instruction and data cache modules will be implemented for each core to minimize the latency overhead of shared-memory arbitration and improve data locality.
- **ISA Extension Support:** All pipeline control and data modules will be updated to incorporate the decoding and execution logic required for the RV32M and RV32A instruction sets.
- **Control-Flow Optimization:** A dynamic branch prediction module will be integrated into the instruction fetch stage to minimize and mitigate control-hazard penalties.

7. Demonstration Plan

The functionality and performance of the multi-core SoC will be validated through a rigorous hardware-in-the-loop demonstration using the following methodology:

- **Benchmark Suite:** A series of benchmarked multithreaded algorithms, developed in C/C++, will be cross-compiled for the RISC-V target to stress test parallel execution, cache coherency,



and pipeline efficiency across multiple cores.

- **System Input:** The resulting cross-compiled RISC-V binaries—incorporating specific thread collision logic are loaded directly into the FPGA’s internal BRAM to enable autonomous hardware execution.
- **System Output:** Final computation results, successful mutex lock verifications, and live hardware profiling metrics will be serialized via the memory-mapped UART transmitter to a serial terminal emulator on a connected laptop[cite: 40].



8. High Level Timeline

Week	Milestone & Deliverable
23–29 Mar	Architecture Design & Environment Setup: Finalize specifications for the 5-stage pipeline, multi-core bus arbiter, and L1 cache coherency protocols. Establish Git CI/CD and Verilog templates
30 Mar–5 Apr	Pipeline Refactoring & Hardware Math: Transition the baseline core from a 3-stage to a 5-stage pipeline with hazard detection units. Integrate the <code>mult_div.v</code> (RV32M) unit.
6–12 Apr	Multi-Core Integration & Memory Hierarchy: Implement the centralized <code>bus_arbiter.v</code> and instantiate multiple cores. Develop and integrate independent L1 instruction and data caches.
13–19 Apr	Concurrency (RV32A) & Branch Prediction: Finalize the RV32A atomic instruction logic and exclusive monitors. Integrate dynamic branch prediction logic.
20–26 Apr	SIMD (RV32P) & System Telemetry: Augment the ALU for RV32P SIMD operations. Map the UART transmitter for real-time telemetry and finalize Zicsr performance counters.

9. Team Work Plan

Member	Responsibilities
Abhigyan	Datapath (RV32M & RV32P): Designs the hardware multiplier and re-engineers the ALU for SIMD carry-chain breaks.
Shivansh	Memory & Control (RV32A): Develops the bus arbiter, L1 cache modules, and the RV32A Exclusive Monitor state machine.
Sarvesh	Systems Integration: Handles Vivado FPGA synthesis, BRAM mapping, Clock PLLs, and UART peripheral design.
Manthan	Verification: Writes cross-compiled C-programs for multithreaded benchmarks and handles <code>.vcd</code> waveform timing analysis.

10. Github Link

https://github.com/sarveshjoshi29/processor_hardwarelab